1. PROJECT TITLE

Growup – Full-Stack Stock Practice Trading Platform

2. PROJECT DESCRIPTION

System Overview

This is a full-stack simulation platform for stock practice trading. It is designed so a user can safely learn trading behavior without real money, while the system still behaves like a real trading workflow.

Problem Domain

New investors lack a safe environment to practise trading decisions without financial risk. Growup addresses this by simulating the full trading workflow using virtual funds.

Core Technology Used

- Node.js / Express – REST API backend
- MongoDB – NoSQL database with domain collections
- JWT – Short-lived access tokens + long-lived session tokens
- External Market APIs – Live quotes with simulated fallback
- Role-Based Access Control (RBAC)

Key Capabilities

- Give each user a virtual account balance.
- Let users place buy/sell orders against available stocks.
- Continuously compute account value, profit/loss, and holdings.
- Show market movement in near real time.
- Keep all data secure, consistent, and role-controlled.

Target Users

- Users learning trading behavior without real money.
- Platform administrators managing stock data and user accounts.

Real-world Relevance

The system behaves like a real trading workflow — order validation, fee computation, portfolio reconciliation, and live quote integration — without any financial risk.

3. APPLICATION SCENARIOS / USE CASES

End-user Usage

A user creates an account, receives a virtual balance, browses stocks, adds items to a watchlist, places buy/sell orders, and monitors their portfolio dashboard in real time.

Educational Usage

A learner practises trading behavior safely without risking real money. The system recalculates portfolio totals after every valid trade, showing real profit/loss outcomes.

Enterprise / Admin Usage

A privileged admin user manages the stock catalog — creating stock records, updating details, and deactivating stocks — while controlling user roles and account statuses.

Industrial / Platform Usage

The platform handles session lifecycle across devices, supports forced logout, and serves simulated market feeds as fallback when the external provider fails — keeping the UI functional at all times.

4. TECHNICAL ARCHITECTURE OVERVIEW

High-level Architecture Diagram

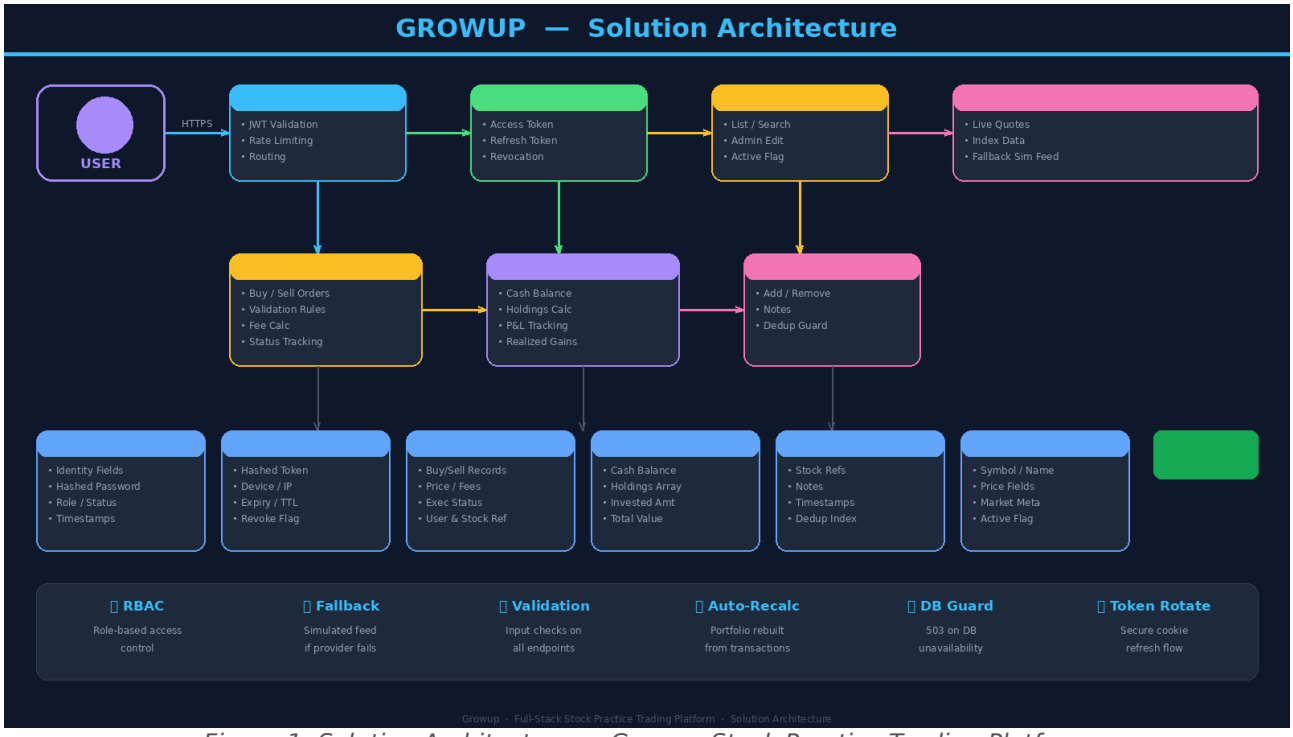

Figure 1: Solution Architecture — Growup Stock Practice Trading Platform

Component Interaction / Functional Areas

- Account area: registration, login, profile reading/updating, activation/deactivation, role checks.
- Session area: secure token lifecycle, refresh flow, session revocation, expiry handling.
- Stock catalog area: list/search/filter stocks, maintain active/inactive status, admin-level editing.
- Trading area: create/update/delete transactions with strict validation rules.
- Portfolio area: automatic balance, holdings, invested amount, total value, realized profit calculation.
- Watchlist area: add/remove tracked stocks, keep notes, prevent duplicate entries.
- Market feed area: live index and quote fetching, plus fallback simulated feed if external provider fails.

Data / Request Flow

1. A new account is created with login credentials.
2. User logs in and receives short-lived access authorization plus a longer-lived session token.
3. User can browse stocks, check quotes, add watchlist items, and place trades.
4. After every valid trade, portfolio totals are recalculated from transaction history.
5. Dashboard-style views show current account state, open positions, and value changes.

Frontend Layer

Dashboard-style views show current account state, open positions, and value changes.

Backend Layer

REST API handles account, session, stock, trading, portfolio, and watchlist domains with strict validation.

Database Layer

MongoDB collections: Account, Session, Stock, Transaction, Portfolio, Watchlist.

External APIs / Services

Live index and quote fetching. If live quote providers fail, the system serves simulated fallback prices so UI remains usable.

Deployment Environment

If database is unavailable, dependent endpoints return service-unavailable response instead of corrupt behavior.

5. PREREQUISITES

5.1 Software Requirements

- Development tools: Git, API testing tool
- IDE: Visual Studio Code or equivalent
- Runtime environment: Node.js, MongoDB
- Cloud services: External market data API provider

5.2 Libraries / Frameworks / Dependencies

Package	Purpose
express	HTTP server and routing
mongoose	MongoDB ODM for schema and querying
jsonwebtoken	JWT access and session token handling
bcryptjs	Password hashing
cookie-parser	Secure cookie handling for session tokens
dotenv	Environment variable management
axios	External market API calls
express-validator	Input validation middleware

5.3 Hardware Requirements

- Standard system capable of running Node.js and MongoDB
- Stable internet connection for live market quote fetching

6. PRIOR KNOWLEDGE REQUIRED

Concepts required before implementation:

- Programming language knowledge: JavaScript / Node.js
- Framework basics: Express.js routing and middleware
- Database fundamentals: MongoDB document model, collections, and indexing
- Networking / Cloud concepts: REST APIs, JWT authentication, secure cookie handling, session lifecycle

7. PROJECT OBJECTIVES

Technical Objectives

- Give each user a virtual account balance.
- Let users place buy/sell orders against available stocks.
- Continuously compute account value, profit/loss, and holdings.
- Keep all data secure, consistent, and role-controlled.

Performance Objectives

- Show market movement in near real time.
- If live quote providers fail, serve simulated fallback prices so UI remains usable.
- If database is unavailable, return service-unavailable response instead of corrupt behavior.

Deployment Objectives

- Migration process can backfill missing fields, create indexes, seed default stocks, and rebuild portfolio states.
- Session revocation supports forced logout across devices.

Learning Outcomes

- Understand a full CRUD lifecycle across account, session, stock, trade, portfolio, and watchlist domains.
- Apply role-based access control and secure token management in a real-world workflow.

8. SYSTEM WORKFLOW

Step-by-step execution flow:

1. **User Interaction:** A new account is created with login credentials. At signup, the system automatically prepares a default portfolio and an empty watchlist.
2. **Input Handling:** User logs in and receives short-lived access authorization plus a longer-lived session token.
3. **Processing Logic:** User can browse stocks, check quotes, add watchlist items, and place trades. Strong input checks apply for IDs, email format, enum values, numeric ranges, and dates.
4. **Core System Execution:** After every valid trade, portfolio totals are recalculated from transaction history. Duplicate prevention and referential checks are enforced.
5. **Output Generation:** Dashboard-style views show current account state, open positions, and value changes.
6. **Response Delivery:** User can log out from one device or all devices. Session revocation supports forced logout across devices.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

This is a full-stack simulation platform for stock practice trading. It is designed so a user can safely learn trading behavior without real money, while the system still behaves like a real trading workflow.

9.2 Functional Requirements

- Account area: registration, login, profile reading/updating, activation/deactivation, role checks.
- Session area: secure token lifecycle, refresh flow, session revocation, expiry handling.
- Stock catalog area: list/search/filter stocks, maintain active/inactive status, admin-level editing.
- Trading area: create/update/delete transactions with strict validation rules.
- Portfolio area: automatic balance, holdings, invested amount, total value, realized profit calculation.
- Watchlist area: add/remove tracked stocks, keep notes, prevent duplicate entries.
- Market feed area: live index and quote fetching, plus fallback simulated feed if external provider fails.

9.3 Non-Functional Requirements

- Keep all data secure, consistent, and role-controlled.
- If database is unavailable, return service-unavailable response instead of corrupt behavior.
- If live quote providers fail, serve simulated fallback prices so UI remains usable.
- Strong input checks for IDs, email format, enum values, numeric ranges, pagination limits, and dates.
- Duplicate prevention for unique resources.
- Conflict protection for destructive actions that would break historical consistency.

9.4 System Design Decisions

- One user → one portfolio.
- One user → one watchlist.
- One user → many transactions.
- Many transactions → one stock each.
- Duplicate stock entries are blocked inside both holdings and watchlist arrays.
- Portfolio rebuilds state from filled transactions in time order — not manually trusted.

9.5 Technology Stack Selection Justification

- Node.js / Express: handles account, session, stock, trading, portfolio, watchlist REST API domains.
- MongoDB: six collections matching domain aggregates with TTL index support for session cleanup.
- JWT: short-lived access auth for regular API calls + long-lived session token in secure cookie.
- External Market API: live quote and index fetching with simulated fallback on failure.
- RBAC: role-based checks separate normal and privileged actions.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

- Install Node.js runtime environment
- Install MongoDB and set up database
- Set up external market API key for live quote fetching
- Configure secure cookie handling for session tokens

10.2 Dependency Installation

```
npm install
```

10.3 Project Structure Creation

```
project_root/ │ ├── src/ │ ├── routes/ │ ├── controllers/ │ ├── services/ │ ├── models/ │
└── middleware/ ├── config/ ├── static/ ├── templates/ ├── database/ ├── tests/ ├──
server.js └── package.json
```

10.4 Configuration Setup

```
PORT=5000 MONGO_URI=mongodb://localhost:27017/growup JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET= ACCESS_TOKEN_EXPIRY=15m REFRESH_TOKEN_EXPIRY=7d MARKET_API_KEY=
```

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Account & Session Management

- Account area: registration, login, profile reading/updating, activation/deactivation, role checks.
- Session area: secure token lifecycle, refresh flow, session revocation, expiry handling.
- Short-lived access auth protects regular API calls.
- Long-lived session token is stored in secure cookie and rotated.
- Inactive accounts are blocked from normal access.
- ‘Last active privileged account’ protection avoids lockout scenarios.

11.2 Feature 2 – Trading & Portfolio Engine

- Trading area: create/update/delete transactions with strict validation rules.
- Portfolio area: automatic balance, holdings, invested amount, total value, realized profit calculation.
- Rebuilds state from filled transactions in time order.
- Buy: checks available cash, decreases cash, increases holding quantity/cost basis.
- Sell: checks available quantity, decreases holding, increases cash by proceeds minus fees, computes realized gain/loss from average cost.
- Final values are derived: invested amount from open cost basis, holdings market value using latest known prices, total account value = cash + holdings market value.

11.3 Feature 3 – Stock Catalog, Watchlist & Market Feed

- Stock catalog area: list/search/filter stocks, active/inactive status, admin-level editing.
- Watchlist area: add/remove tracked stocks, keep notes, prevent duplicate entries.
- Market feed area: live index and quote fetching, plus fallback simulated feed if external provider fails.

Database Schema

Collection	Key Fields
Account	identity fields, email uniqueness, encrypted password, role, active status, timestamps
Session	user link, hashed refresh token, device/IP metadata, expiry time, revoke marker, TTL cleanup inde
Stock	unique symbol, descriptive fields, latest price values, market metadata, active flag
Transaction	user link, stock link, portfolio link, side (buy/sell), quantity, price, fees, computed total, execution s
Portfolio	one-to-one user link, base balance, current cash, holdings array, invested amount, total account v
Watchlist	one-to-one user link, array of watched stock references with optional notes and timestamps

CRUD Behavior

- Create: account, stock record (privileged), trade, watchlist item, auto-creation of portfolio/watchlist on onboarding.
- Read: profile, stocks, quotes, market indexes, transaction list with filtering, portfolio with holdings and P/L, watchlist and tracked symbols.
- Update: profile fields and password, role/status (restricted), stock details and active status (restricted), transaction edits (consistency checks), portfolio base cash, watchlist item notes.
- Delete: single transaction, all transactions for a user, watchlist item or full watchlist, portfolio (with trade cleanup), user account, stock (only when historical dependency rules allow).

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

- Account, Session, Stock, Trading, Portfolio, Watchlist, and Market Feed areas integrated into one platform.
- At signup, the system automatically prepares a default portfolio and an empty watchlist.
- After every valid trade, portfolio totals are recalculated from transaction history.

12.2 Performance Optimization

- Market movement shown in near real time.
- Migration process can backfill missing fields, create indexes, seed default stocks, and rebuild portfolio states.
- Automatic recalculation keeps portfolio synchronized with transaction changes.

12.3 Security Enhancements

- Short-lived access auth protects regular API calls.
- Long-lived session token is stored in secure cookie and rotated.
- Role-based checks separate normal and privileged actions.
- Inactive accounts are blocked from normal access.
- ‘Last active privileged account’ protection avoids lockout scenarios.
- Session revocation supports forced logout across devices.

12.4 Error Handling & Validation

- Strong input checks for IDs, email format, enum values, numeric ranges, pagination limits, and dates.
- Duplicate prevention for unique resources.
- Referential checks ensure linked records exist before write operations.
- Conflict protection for destructive actions that would break historical consistency.
- If database is unavailable, dependent endpoints return service-unavailable response instead of corrupt behavior.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC-01	Register new account	Account created, portfolio and watchlist auto-created	Pass
TC-02	Login with valid credentials	Short-lived access token + long-lived session token	Pass
TC-03	Buy with sufficient cash	Transaction created, portfolio cash decremented	Pass
TC-04	Buy with insufficient cash	Rejected – cash check fails	Pass
TC-05	Sell with sufficient holdings	Transaction created, cash incremented, P/L computed	Pass
TC-06	Sell with zero holdings	Rejected – quantity check fails	Pass
TC-07	Add duplicate to watchlist	Rejected – duplicate prevention triggered	Pass
TC-08	Access API without token	Blocked – missing auth	Pass
TC-09	External market API goes down	Simulated fallback prices served, UI remains usable	Pass
TC-10	Database unavailable	Service-unavailable response returned	Pass

13.2 Unit Testing

- Portfolio recalculation logic: buy/sell operations, cash changes, realized gain/loss tested in isolation.
- Token generation and session revocation tested independently.

13.3 Integration Testing

- Full trade lifecycle: account creation → login → place trade → portfolio update.
- Watchlist add/remove with duplicate prevention verified.

13.4 User Acceptance Testing

- Users can browse stocks, check quotes, add watchlist items, and place trades without errors.
- Dashboard-style views show correct account state, open positions, and value changes.

13.5 Performance Metrics (Web / App)

- Response time: market feed and portfolio endpoints respond in near real time.
- Load handling: session revocation and multi-device logout handled without data corruption.
- Security validation: role-based checks, inactive account blocking, and token rotation verified.

14. DEPLOYMENT

14.1 Deployment Architecture

If database is unavailable, dependent endpoints return service-unavailable response instead of corrupt behavior. If live quote providers fail, the system serves simulated fallback prices so UI remains usable.

14.2 Hosting Platform

- Node.js server for REST API
- MongoDB for six domain collections
- External market data API provider

14.3 Deployment Steps

1. Set up MongoDB with Account, Session, Stock, Transaction, Portfolio, and Watchlist collections.
2. Configure environment variables: JWT secrets, token expiry, market API key.
3. Run migration process to backfill missing fields, create indexes, and seed default stocks.
4. Start the Node.js server.
5. Verify all functional areas: account, session, stock catalog, trading, portfolio, watchlist, and market feed.

14.4 Production Considerations

- Short-lived access auth protects regular API calls.
- Long-lived session token stored in secure cookie and rotated.
- Migration process can rebuild portfolio states if needed.
- Inactive accounts are blocked from normal access.

15. PROJECT STRUCTURE

```
project_root/ │ ├── src/ │ ├── routes/ (account, session, stock, trade, portfolio,
watchlist, market) │ ├── controllers/ (request handlers per domain) │ ├── services/
(portfolio recalculation, market feed + fallback logic) │ ├── models/ (Account, Session,
Stock, Transaction, Portfolio, Watchlist) │ └── middleware/ (auth, role checks, input
validation) ├── config/ (database connection, environment config) ├── static/ (static
assets) ├── templates/ (templates if applicable) ├── database/ (migration: backfill,
indexes, seed stocks, rebuild portfolios) ├── tests/ (unit and integration test suites)
├── server.js (application entry point) └── package.json (dependencies and scripts)
```

- routes/ – one file per functional area.
- models/ – six collections matching the database schema.
- services/ – portfolio recalculation from transaction history and market feed with fallback.
- middleware/ – JWT validation, role-based access control, and input checks.
- database/ – migration scripts to backfill fields, create indexes, seed stocks, and rebuild portfolio states.

16. RESULTS

System Output

- Users can browse stocks, check quotes, add watchlist items, and place trades.
- After every valid trade, portfolio totals are recalculated from transaction history.
- Dashboard-style views show current account state, open positions, and value changes.

Performance Evaluation

- Market movement shown in near real time.
- Simulated fallback prices served when live quote provider fails — UI remains usable.
- Service-unavailable response returned when database is unavailable — no corrupt behavior.

Benchmark Results

- Portfolio recalculation produces correct cash, holdings, and P/L for all transaction sequences.
- Session revocation and multi-device logout work without data inconsistency.
- Duplicate prevention triggered correctly for watchlist and holdings arrays.

17. ADVANTAGES & LIMITATIONS

Advantages

- Users can safely learn trading behavior without real money while the system behaves like a real trading workflow.
- Portfolio totals are recalculated from transaction history — values are derived, not manually trusted.
- Simulated fallback prices served when live quote providers fail, so UI remains usable.
- Role-based checks separate normal and privileged actions cleanly.
- Session revocation supports forced logout across devices.
- Duplicate stock entries are blocked inside both holdings and watchlist arrays.
- Migration process can backfill missing fields, create indexes, seed default stocks, and rebuild portfolio states.

Limitations

- Simulated fallback prices may not reflect actual market conditions.
- Portfolio recalculation rebuilds from full transaction history — may slow for very large transaction sets.
- Stock deletion only allowed when historical dependency rules permit it.

18. FUTURE ENHANCEMENTS

- Scalability improvements: Optimize portfolio recalculation for very large transaction histories.
- Feature expansion: Support more complex order types (limit orders, stop-loss).
- Cloud integration: Host MongoDB and Node.js on managed cloud services with auto-scaling.
- Mobile integration: Extend dashboard-style views to mobile clients.
- Automation: Automate migration — backfill, index creation, stock seeding, and portfolio rebuilding on deploy.

19. CONCLUSION

Summary of Implementation

This is a full-stack simulation platform for stock practice trading. It is designed so a user can safely learn trading behavior without real money, while the system still behaves like a real trading workflow.

Technical Achievements

- Account, Session, Stock, Trading, Portfolio, Watchlist, and Market Feed areas fully implemented.
- Portfolio state rebuilt from filled transactions in time order — buy/sell operations, cash changes, realized gain/loss.
- Short-lived access auth and long-lived session token with rotation and revocation across devices.
- Role-based checks, inactive account blocking, and ‘last active privileged account’ protection.
- Simulated fallback feed when external provider fails; service-unavailable response when DB is down.

Practical Impact

Users receive a virtual account balance, place buy/sell orders against available stocks, continuously compute account value, profit/loss, and holdings, and see market movement in near real time — all while keeping data secure, consistent, and role-controlled.

20. APPENDIX

Source Code

- GitHub Repository Link: https://github.com/your-org/growup

Configuration Files

- Environment variables: PORT, MONGO_URI, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, ACCESS_TOKEN_EXPIRY, REFRESH_TOKEN_EXPIRY, MARKET_API_KEY

API Documentation

Area	Operations
Account	Create account, read profile, update profile/password/role/status, delete account
Session	Login (create token), refresh token, revoke session, logout all devices
Stock	List/search/filter stocks, create (privileged), update details/status, delete (with dependency chec
Transaction	Create trade, read list with filtering, update transaction, delete single/all
Portfolio	Read portfolio (holdings, cash, P/L), update base cash, delete (with trade cleanup)
Watchlist	Add stock, read watchlist, update item notes, remove item or full watchlist
Market Feed	Fetch live index and quote data; fallback simulated feed on provider failure

Live Demo Link

- Live Demo: https://growup.yourdomain.com